

```
-----  
-----  
-- Engineer: Olasubomi Borishade  
--  
-- Create Date: 06/21/2026 04:51:45 PM  
-- Design Name: Matrix Accelerator  
-- Module Name: Matrix_Accelerator - Behavioral  
-- Project Name: Parameterized Matrix Multiply Engine  
  
-- Description: Top-level matrix accelerator that stores two input  
matrices,  
--               streams them into a systolic array on Start, and  
asserts Done  
--               when the output matrix is ready to be read.  
  
-- Revision:  
-- Revision 0.01 - File Created  
-- Additional Comments:  
--  
-----  
-----
```

```
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
use WORK.ARRAY_CUSTOM_PACK.ALL;  
use IEEE.NUMERIC_STD.ALL;  
  
-- Uncomment the following library declaration if instantiating  
-- any Xilinx leaf cells in this code.  
--library UNISIM;  
--use UNISIM.VComponents.all;  
  
entity Matrix_Accelerator is  
    Generic (  
        Rows          : INTEGER := 2;  
        Cols           : INTEGER := 2;  
        Input_Width    : INTEGER := 8;
```

```

        Accumulator_Width : INTEGER := 32
    );

    Port      (
        Clock      : in  STD_LOGIC;
        Reset      : in  STD_LOGIC;
        Start      : in  STD_LOGIC;
        Done       : out STD_LOGIC;
        A_Write_En  : in  STD_LOGIC;
        A_Write_Row : in  INTEGER range 0 to Rows -
1;
        A_Write_Col : in  INTEGER range 0 to Cols -
1;
        A_Write_Data : in  SIGNED(Input_Width - 1
downto 0);
        B_Write_En  : in  STD_LOGIC;
        B_Write_Row : in  INTEGER range 0 to Rows -
1;
        B_Write_Col : in  INTEGER range 0 to Cols -
1;
        B_Write_Data : in  SIGNED(Input_Width - 1
downto 0);
        Output      : out SignedArray_2d(0 to Rows -
1)(0 to Cols - 1)(Accumulator_Width - 1 downto 0)
    );

```

end Matrix_Accelerator;

architecture Behavioral of Matrix_Accelerator is

Type State_Type is (IDLE, LOAD, DRAIN, VALID);

Signal State : State_Type := IDLE;

Signal A_Reg : SignedArray_2d(0 to Rows - 1)(0 to Cols - 1)(Input_Width - 1 downto 0);

Signal B_Reg : SignedArray_2d(0 to Rows - 1)(0 to Cols - 1)(Input_Width - 1 downto 0);

Signal Cycle_Count : INTEGER := 0;

```

Signal SA_Enable      : STD_LOGIC := '0';
Signal SA_Input_A     : SignedArray_1d(0 to Rows - 1)(Input_Width - 1
downto 0);
Signal SA_Input_B     : SignedArray_1d(0 to Cols - 1)(Input_Width - 1
downto 0);
Signal SA_Output      : SignedArray_2d(0 to Rows - 1)(0 to Cols -
1)(Accumulator_Width - 1 downto 0);
Signal Output_Reg     : SignedArray_2d(0 to Rows - 1)(0 to Cols -
1)(Accumulator_Width - 1 downto 0);

```

```

begin

```

```

SA_Inst : entity work.Systolic_Array(Behavioral)

```

```

    Generic Map (      Rows          => Rows,
                       Cols          => Cols,
                       Input_Width   => Input_Width,
                       Accumulator_Width => Accumulator_Width
                       )

```

```

    Port Map      (      Input_A      => SA_Input_A,
                       Input_B      => SA_Input_B,
                       Clock        => Clock,
                       Reset        => Reset,
                       Enable       => SA_Enable,
                       Clear_Accumulator => Start,
                       Output       => SA_Output
                       );

```

```

process(Clock)

```

```

begin

```

```

    if rising_edge(Clock) then
        if A_Write_En = '1' then
            A_Reg(A_Write_Row)(A_Write_Col)
                <= A_Write_Data;

```

```

        end if;
        if B_Write_En = '1' then
            B_Reg(B_Write_Row) (B_Write_Col)
                <= B_Write_Data;
        end if;
    end if;
end process;

process(Clock)
begin
    if rising_edge(Clock) then
        if Reset = '1' then
            State <= IDLE;
            Cycle_Count <= 0;
        else
            case State is
                when IDLE =>
                    if Start = '1' then
                        State <= LOAD;
                        Cycle_Count <= 0;
                    end if;
                when LOAD =>
                    if Cycle_Count = Cols - 1 then
                        State <= DRAIN;
                        Cycle_Count <= 0;
                    else
                        Cycle_Count <= Cycle_Count + 1;
                    end if;
                when DRAIN =>
                    if Cycle_Count = Rows + Cols - 2 then
                        Output_Reg <= SA_Output;
                        State <= VALID;
                    else
                        Cycle_Count <= Cycle_Count + 1;
                    end if;
                when VALID =>

```

```

        if Start = '0' then
            State <= IDLE;
        end if;
    end case;
end if;
end if;
end process;

process(all)
begin
    SA_Input_A <= (others => (others => '0'));
    SA_Input_B <= (others => (others => '0'));
    SA_Enable <= '0';

    if State = LOAD or State = DRAIN then
        SA_Enable <= '1';
    end if;

    if State = LOAD then
        for r in 0 to Rows - 1 loop
            SA_Input_A(r) <= A_Reg(r)(Cycle_Count);
        end loop;

        for c in 0 to Cols - 1 loop
            SA_Input_B(c) <= B_Reg(Cycle_Count)(c);
        end loop;
    end if;
end process;

Output <= Output_Reg;

Done <= '1' when State = VALID else '0';

end Behavioral;

```